Trust & Provenance Layer for ARQ

Normative Specification

Kotov Ivan

2026, Bruxelles

Contents

T1. Hardware Attestation . . . 1
T2. Anomaly Prior and Noise Model Governance . . . 3
T3. Signed Controller State and Witness Integrity . . . 4
T4. Interaction with Privilege Policy . . . 4
T5. The No-Value-Without-Witness Principle . . . 5
T6. Implementation Guidance (Non-Normative) . . . 5
Author . . . 6

This document defines the mandatory trust and provenance mechanisms that supplement the ARQ protocol. Its goal is simple: no error event may be treated as valuable unless it is authentic, attributable, and bound to a concrete witness trail.

T1. Hardware Attestation

T1.1 Attestation Report Format

At the beginning of each **calibration epoch** (`T_epoch`, configurable by `a`), every component involved in error detection and correction (for example, quantum processor, pulse generator, readout chain, or trusted execution environment) must produce a signed attestation report in canonical JSON.

The report contains the following mandatory fields:

```
{
  "version": "1.0",
  "component_id": "<unique device identifier>",
  "firmware_version": "<semver>",
  "calibration_epoch": <uint64>,
  "timestamp": "<ISO 8601 UTC>",
  "validity_interval": {
    "start": "<ISO 8601 UTC>",
```

```
    "end": "<ISO 8601 UTC>"
  },
  "calibration_parameters": {
    "noise_model": "<hash of noise model blob>",
    "coherence_times": { "T1": <float>, "T2": <float> },
    "gate_fidelities": {
      "single_qubit": <float>,
      "two_qubit": <float>
    },
    "resonance_frequencies": [<float>],
    "temperature": <float>,
    "voltage": <float>,
    "clock_drift_ppm": <float>
  },
  "trust_anchor": "<public key fingerprint of root of trust>",
  "signature": "<base64 encoded signature>"
}
```

Canonicalization. The JSON is serialized without whitespace, keys are sorted lexicographically, and the payload is UTF-8 encoded. The hash is computed with SHA-256 over the canonical representation *before* the `signature` field is added. The signature is then appended as a top-level field and covers the canonical payload including the hash of the preceding attestation chain.

T1.2 Epoch Validity

- An epoch is **valid** if the current time is within the `validity_interval` **and** no invalidation event has occurred.
- The epoch length `T_epoch` is set by `a` and recorded in the witness log.
- Typical values are 1 hour for stable lab environments and 10 minutes for field deployments.

T1.3 Early Invalidation

An epoch is **invalidated immediately** if any of the following occur:

- **Hardware reset** (power cycle, watchdog reboot) - triggers new calibration.
- **Uncorrectable deviation** - measured environmental parameters exceed thresholds defined in the calibration policy (for example, temperature drift > 2 deg C, clock drift > 10 ppm).
- **Failed verification** - a subsequent attestation report cannot be verified against the chain of trust.
- **Operator-issued halt** - `a` may explicitly invalidate the epoch via a signed witness entry.
- **Exhaustion of privilege budget** for that epoch (see T4).

When invalidation occurs, the system enters a **safe state** (fail-closed) and refuses to process further error events until a new attestation report is accepted.

T1.4 Recalibration and Trust Rotation

Recalibration is the process of obtaining a new attestation report with a fresh `calibration_epoch` incremented by 1. The new report must be signed with the same root of trust, or a successor if a

trust handover is performed. After a successful recalibration, the previous epoch is archived and can no longer be used to validate events. The system then resumes normal operation.

Recalibration can be triggered:

- Automatically at epoch expiry.
- By operator request.
- Automatically when early invalidation conditions are met.

All recalibration events are recorded in the witness chain with full details, including the old epoch hash, the new report, and the reason.

T2. Anomaly Prior and Noise Model Governance

T2.1 Conservative Baseline

The **background probability** $p_{\text{background}}$ is **not** a free parameter. It is defined as a conservative upper bound on the probability that an observed deviation could be produced by any *uncontrolled* process, including adversarial activity, given the current attestation. It is set by `a` as a **policy constant** and cannot be modified by the entity `c` without a witness-challengeable operator override.

In the absence of a specific adversarial model, $p_{\text{background}}$ defaults to:

```
p_background = 10^-3
```

This value is chosen to be several orders of magnitude larger than a typical calibrated noise floor (often below 10^{-6}) while still keeping anomalies rare.

T2.2 Noise Model and Its Limitation

The calibrated noise model $\mathcal{M}$ is derived from the attestation report's `calibration_parameters` and is used to compute $p(\text{event} \mid \mathcal{M})$. It is **never** used alone to decide the value of an error; it is always combined with the background model.

To prevent the noise model from becoming a self-justification tool:

- The noise model is frozen for the duration of an epoch; it cannot be updated within the epoch.
- Any discrepancy between the model and observed statistics that exceeds a threshold (for example, 5%) triggers early invalidation.
- The anomaly prior formula is **hard-coded**:

$$P_{\text{anom}} = 1 - \frac{p(\text{event} \mid \mathcal{M})}{p(\text{event} \mid \mathcal{M}) + p_{\text{background}}}$$

where $p(\text{event} \mid \mathcal{M})$ is the probability density of the observed deviation under the calibrated model. If $p(\text{event} \mid \mathcal{M})$ is smaller than the numerical precision, it is clamped to a minimum of 10^{-12} to avoid division issues.

T2.3 Challengeability of the Baseline

If `a` believes that the conservative baseline is too strict, or too loose, a **challenge** may be initiated through the witness system. The challenge must include:

- an alternative $p_{\text{background}}$ value, and
- a justification, such as new empirical data.

The change is accepted only if it is signed by `a`, recorded in the witness log, and applied **only to future epochs**.

T3. Signed Controller State and Witness Integrity

T3.1 Controller Log

Every action taken by the controller that affects the system state - including anti-resonance pulses, syndrome measurements, or EA minting - is recorded in a signed log.

Each log entry contains:

- `timestamp` (monotonic, hardware-synchronized),
- `action_type` (for example, `pulse`, `measurement`, `ea_mint`),
- `parameters` (for example, pulse shape, qubit indices, EA hash),
- `epoch_id` (the current calibration epoch),
- `signature` over the canonical entry.

The log is appended to the same hash chain used for ARQ capsules, ensuring a single tamper-evident timeline.

T3.2 Assumption-Bound Witness

Each ARQ capsule's `trust_context` includes the `attestation_hash` and `calibration_epoch`. During audit, an external verifier can:

- retrieve the attestation report for that epoch,
- verify its signature and confirm that the epoch was valid at the time,
- confirm that the capsule's `witness_hash` matches the log entry,
- re-compute P_{anom} using the archived noise model.

If any step fails, the capsule is considered **untrustworthy**, and its contents cannot be used for EA minting or any other privileged action.

T4. Interaction with Privilege Policy

T4.1 Trust-Sensitive Privilege Envelope

The privilege envelope - the set of actions that `c` is allowed to perform - is **scaled by trust weight**. Let τ be defined as in Section 5.5:

$$\tau = \text{attestation_valid} \times (1 - P_{\text{anom}})$$

where `attestation_valid` is 1 if the event's epoch is valid and all signatures verify, and 0 otherwise.

Rule. For any action that requires a privilege (for example, applying an anti-resonance pulse, minting an EA, or invoking an oracle), the entity must have a **remaining privilege budget** that is at least privilege_required/τ, or the action is denied.

If $\tau < \tau_{\min}$ (for example, `0.5`), **all privileged actions are blocked**, and the system enters a safe state until recalibration.

T4.2 Double Signature for High-Value EA Minting

If a capsule's modified value V' exceeds a **high threshold** V_{high} (set by `a`), minting the EA requires **two signatures**:

- the entity `c` signature, as usual,
- a second signature from the human anchor `a` (or a designated delegate) explicitly approving the promotion.

This second signature is recorded in the witness trail and becomes part of the EA. Without it, the capsule remains in `log_only` state.

T5. The No-Value-Without-Witness Principle

Normative statement. An error event shall **not** be considered valuable - that is, it shall not be promoted to an EA - unless it is accompanied by a complete, replayable, and attributable witness trail that satisfies all of the following:

1. **Replayable** - all raw data (measurement outcomes, pulse sequences, controller logs) are archived so that an independent observer can reproduce the error calculation.
2. **Challengeable** - the event is recorded in a tamper-evident hash chain, and any party with appropriate privileges can dispute the classification.
3. **Attributable** - every component involved in the event is identified through a valid attestation report at the time of the event.
4. **Bound to a Witnessable Boundary** - the error occurred within a controlled domain where physical and cryptographic boundaries (L4) were enforced, such as a secure hardware module, an attested quantum processor, or a local trusted execution environment.

Any capsule that does not meet these criteria, even if its computed V' is high, **must** be treated as `log_only` (or suppressed) and **must not** be used to update the entity's long-term memory.

T6. Implementation Guidance (Non-Normative)

- Attestation reports should be stored in a local, append-only directory with file-level checksums.
- The controller log can be implemented as a SQLite database with each row signed using Ed25519.

- The witness chain can be anchored to a Merkle tree, with periodic root hashes published to a public ledger if needed.
- The anomaly prior computation should use double-precision floating point and be unit-tested against known noise distributions.

Author

Kotov Ivan
2026, Bruxelles